CAHIER DES CHARGES

Projet **Larablog**

Projet	Larablog — Plateforme de blog multi-utilisateurs
Cadre	Travail Pratique — BTS SIO option SLAM
Framework	Laravel 10.x + Laravel Breeze + Tailwind CSS
Version	1.0 — Document de référence
Date	2025
Auteur	Théo

Sommaire

3.1 Contexte et objectifs
3.2 Description du produit
3.3 Périmètre fonctionnel
3.4 Exigences fonctionnelles
3.5 Exigences non fonctionnelles
3.6 Contraintes techniques
3.7 Personas / Utilisateurs cibles
3.8 Critères d'acceptation
3.9 Livrables attendus
3.10 Planning estimatif
3.11 Contraintes générales et évolutions

3.1 Contexte et objectifs

Contexte

Larablog est un projet de développement web réalisé dans le cadre d'un travail pratique (TP) de BTS SIO option SLAM. Il s'agit d'une plateforme de blog multi-utilisateurs développée avec le framework Laravel 10 et le kit d'authentification Laravel Breeze. Ce projet vise à mettre en application l'ensemble des compétences acquises en développement back-end et front-end, notamment la gestion des modèles MVC, les relations Eloquent, la sécurité CSRF et la mise en page responsive avec Tailwind CSS.

Objectifs

- Offrir une plateforme de publication d'articles accessible à tout utilisateur inscrit.
- Permettre la gestion complète du cycle de vie d'un article : création, modification, publication et suppression.
- Favoriser l'interaction entre les utilisateurs via les commentaires et les likes.
- Organiser le contenu grâce à un système de catégorisation (et de tags, préparé pour une version future).
- Fournir un accès public clair aux contenus publiés, depuis la page d'accueil jusqu'à la page auteur.

3.2 Description du produit

Larablog est une application web de type blog communautaire permettant à chaque utilisateur inscrit de publier des articles avec un système de brouillon, de classer ses articles par catégories, de recevoir des commentaires et des likes de la part d'autres utilisateurs, et de disposer d'un profil public présentant l'ensemble de ses publications.

Les visiteurs non inscrits bénéficient d'un accès en lecture seule : ils peuvent consulter les articles publiés, parcourir les profils d'auteurs et découvrir les articles les plus populaires depuis la page d'accueil. Les fonctionnalités d'interaction (like, commentaire) sont réservées aux utilisateurs authentifiés.

3.3 Périmètre fonctionnel

Module	Fonctionnalités	Statut
Authentification	Inscription, connexion, déconnexion, réinitialisation du mot de passe	✅ Opérationnel
Profil utilisateur	Modification des informations personnelles, changement de mot de passe, suppression de compte	✅ Opérationnel
Gestion des articles	Création, modification, suppression, publication / brouillon	✅ Opérationnel
Catégorisation	Association d'articles à des catégories — relation many-to-many via table pivot	✅ Opérationnel
Tags	Modèle et table pivot préparés — non exploités dans les vues actuelles	⏳ Prévu v2
Commentaires	Ajout de commentaires par les utilisateurs authentifiés, affichage avec auteur et date	✅ Opérationnel
Likes	Compteur de likes incrémentable — affiché sur la home, le dashboard et la page article	✅ Opérationnel

Module	Fonctionnalités	Statut
Consultation publique	Page d'accueil, profil auteur, lecture d'article — accessibles sans inscription	✓Opérationnel

3.4 Exigences fonctionnelles

EF-01 — Authentification

- Le système doit permettre l'inscription avec nom, adresse e-mail et mot de passe (confirmation obligatoire).
- Le système doit permettre la connexion avec e-mail et mot de passe, avec option « Se souvenir de moi ».
- Le système doit permettre la déconnexion depuis n'importe quelle page via la barre de navigation.
- Un mécanisme de réinitialisation de mot de passe par lien e-mail doit être disponible.
- La protection CSRF doit être appliquée sur l'ensemble des formulaires de l'application.

EF-02 — Gestion du profil utilisateur

- L'utilisateur authentifié peut modifier son nom et son adresse e-mail.
- La modification de l'adresse e-mail réinitialise le statut de vérification (email_verified_at).
- L'utilisateur peut modifier son mot de passe via un formulaire dédié (hashage Bcrypt automatique).
- L'utilisateur peut supprimer définitivement son compte après confirmation par saisie de son mot de passe actuel.

EF-03 — Gestion des articles

- Un utilisateur authentifié peut créer un article avec un titre et un contenu.
- Un article peut être enregistré en brouillon (draft = 1) ou publié (draft = 0).
- Un article en brouillon n'est jamais affiché sur les pages publiques.
- Seul le propriétaire d'un article peut le modifier ou le supprimer.
- La suppression d'un article entraîne la suppression en cascade des commentaires associés et le détachement des catégories liées.

EF-04 — Catégories

- Les catégories existent indépendamment des articles et sont gérées en base de données.
- Un article peut être associé à une ou plusieurs catégories (relation many-to-many).
- Les catégories sélectionnées sont synchronisées via la table pivot article_category (méthode sync()).
- Les catégories d'un article sont affichées sur sa page publique de lecture.

EF-05 — Tags (version future)

- Les tags sont modélisés (table tags, table pivot article_tag) mais non exploités dans l'interface actuelle.
- Cette fonctionnalité est planifiée pour une version ultérieure du projet.

EF-06 — Commentaires

- Un utilisateur authentifié peut rédiger et publier un commentaire sur n'importe quel article.
- Chaque commentaire est associé à son auteur (utilisateur) et à l'article commenté.
- Les commentaires sont affichés sur la page de l'article avec le nom de l'auteur et la date relative.
- Un visiteur non connecté voit un message l'invitant à se connecter à la place du formulaire de commentaire.

EF-07 — Likes

- Un utilisateur authentifié peut liker un article en cliquant sur le bouton « J'aime ».

- Chaque clic incrémente de 1 le compteur likes de l'article en base de données.
- Le nombre de likes est affiché sur la page d'accueil, le tableau de bord et la page de l'article.
- Note : l'implémentation actuelle n'impose pas d'unicité du like par utilisateur — évolution prévue en v2.

EF-08 — Pages publiques

- La page d'accueil (/) affiche les 3 articles non-brouillons ayant le plus de likes (tri décroissant).
- La page profil auteur (/{user}) affiche tous les articles publiés d'un utilisateur donné.
- La page article (/{user}/{article}) affiche le contenu complet, les catégories, les commentaires et le nombre de likes.

3.5 Exigences non fonctionnelles

#	Exigence	Description
ENF-01	**Sécurité**	Protection CSRF sur tous les formulaires. Hashage des mots de passe via Bcrypt (intégré Laravel). Vérification de propriété systématique avant toute modification ou suppression d'article.
ENF-02	**Performance**	La page d'accueil ne charge que 3 articles. Toutes les requêtes passent par l'ORM Eloquent, optimisé pour éviter les requêtes N+1 (eager loading via with()).
ENF-03	**Ergonomie**	Interface responsive via Tailwind CSS. Messages flash de succès/erreur après chaque action. Confirmation JavaScript avant toute suppression.
ENF-04	**Accessibilité**	Labels associés à chaque champ de formulaire. Hiérarchie sémantique des titres respectée (h1 > h2 > h3). Contraste suffisant en mode clair et sombre.
ENF-05	**Maintenabilité**	Architecture MVC de Laravel strictement respectée. Séparation claire Controllers / Models / Views. Modèles générés avec Reliese pour une cohérence maximale avec le schéma BDD.
ENF-06	**Compatibilité**	Compatible avec les navigateurs modernes : Chrome, Firefox, Edge, Safari. Support natif du mode sombre (dark mode) via Tailwind.

3.6 Contraintes techniques

Contrainte	Détail
Framework principal	Laravel 10.x
Kit authentification	Laravel Breeze (scaffolding Blade + Tailwind)
Langage serveur	PHP 8.1 minimum (requis par Laravel 10)
Base de données	MySQL via Laragon (environnement de développement local Windows)
ORM	Eloquent — modèles générés par Reliese Model pour cohérence schéma / code
CSS / UI	Tailwind CSS (classes utilitaires, dark mode intégré)
Build front-end	Vite.js (bundler rapide, HMR en développement)
Serveur local	Laragon (Apache/Nginx + PHP + MySQL — environnement portable Windows)
Protection API	Laravel Sanctum (tokens API — préparé pour usage futur)
Gestion du code	Git (versionnement du code source)

3.7 Personas / Utilisateurs cibles

Trois profils types ont été identifiés pour cadrer les choix de conception fonctionnelle :

Persona 01 — Étudiant blogueur (Utilisateur principal)	
Nom / Âge	Alex, 22 ans
Profil	Étudiant en informatique souhaitant partager ses connaissances techniques.
Besoins	Publier des articles structurés, organiser par catégorie, recevoir des retours concrets (likes, commentaires).
Motivation	Construire un portfolio de publications, accroître sa visibilité auprès de la communauté.
Frustrations	Interface complexe, perte de brouillons non sauvegardés, manque de retours.

Persona 02 — Lecteur occasionnel (Visiteur)	
Nom / Âge	Marie, 30 ans
Profil	Professionnelle curieuse de découvrir des articles techniques sans s'inscrire.
Besoins	Parcourir les articles populaires, lire les contenus, découvrir de nouveaux auteurs.
Motivation	Rester informée, trouver du contenu de qualité rapidement sans friction.
Frustrations	Obligation d'inscription pour accéder à certains contenus, manque de contenu récent.

Persona 03 — Contributeur actif (Utilisateur engagé)	
Nom / Âge	Thomas, 25 ans
Profil	Développeur passionné qui lit, commente et like régulièrement les articles des autres.
Besoins	Interagir avec les auteurs, exprimer son avis via commentaires et likes, suivre les publications.
Motivation	Participer à une communauté active, encourager les auteurs de qualité.
Frustrations	Impossibilité de suivre un auteur, absence de notification de réponse à un commentaire.

3.8 Critères d'acceptation

#	Critère	Validation
CA-01	Un visiteur peut s'inscrire et se connecter.	✅Formulaires Breeze fonctionnels
CA-02	Un utilisateur connecté peut créer, modifier et supprimer ses articles.	✅CRUD complet avec vérification de propriété
CA-03	Un article en brouillon n'apparaît pas sur les pages publiques.	✅Filtre draft = 0 appliqué sur toutes les vues publiques
CA-04	Un utilisateur peut associer des catégories à un article.	✅Checkboxes + sync() — relation pivot correcte
CA-05	Le tableau de bord affiche tous les articles de l'utilisateur.	✅Avec statut, extrait, nombre de likes et actions
CA-06	La page d'accueil affiche les 3 articles les plus likés.	✅Tri par likes DESC, limit(3) appliqué
CA-07	Un utilisateur connecté peut commenter un article.	✅Formulaire fonctionnel + création du modèle Comment
CA-08	Un utilisateur connecté peut liker un article.	✅Incrémentation du compteur en BDD
CA-09	La suppression d'un article supprime ses commentaires et détache ses catégories.	✅detach() + delete() en cascade
CA-10	Un utilisateur peut modifier son profil et supprimer son compte.	✅ProfileController opérationnel

3.9 Livrables attendus

#	Livrable	Description
L-01	**Code source**	Dépôt Git contenant l'intégralité du code Laravel (controllers, models, views, migrations, seeders).
L-02	**Base de données**	Fichiers de migrations et seeders permettant de recréer la structure et les données de test.
L-03	**Documentation**	Cahier des charges (ce document), analyse fonctionnelle (cas d'utilisation, diagrammes UML, schéma E/A).
L-04	**Application**	Application complète déployable localement via Laragon, avec instructions d'installation (README).

3.10 Planning estimatif

Phase	Durée estimée	Statut
Phase 1 — Configuration initiale (Laravel, Breeze, BDD)	1 jour	✅ Terminé
Phase 2 — Authentification et gestion du profil utilisateur	1 jour	✅ Terminé
Phase 3 — CRUD articles + système brouillon / publication	2 jours	✅ Terminé
Phase 4 — Catégories et tags (modèles + pivots)	1 jour	✅ Terminé (tags préparés)
Phase 5 — Commentaires	0,5 jour	✅ Terminé
Phase 6 — Likes	0,5 jour	✅ Terminé
Phase 7 — Pages publiques (accueil, profil auteur, article)	1 jour	✅ Terminé
Phase 8 — UI/UX (Tailwind, responsive, dark mode)	1 jour	✅ Terminé
Phase 9 — Tests et corrections	1 jour	⏳ En attente
Phase 10 — Documentation et livraison	0,5 jour	⏳ En cours
Total estimé	**~9,5 jours**	**8/10 phases terminées**

3.11 Contraintes générales et évolutions futures

Contraintes générales

Contrainte	Description
Délai	Projet à rendre dans le cadre d'un TP avec une échéance fixée par l'enseignant.
Environnement	Développement local sous Windows avec Laragon (Apache + PHP + MySQL intégrés).
Langue	Interface utilisateur entièrement en français.
Données personnelles	Projet éducatif sans données sensibles réelles. Les mots de passe sont néanmoins hashés via Bcrypt.

Évolutions identifiées pour les versions futures

- Exploitation des tags dans l'interface (ajout aux formulaires, filtrage, page par tag).
- Pagination des articles sur les pages publiques et le tableau de bord.
- Système de notifications (réponse à un commentaire, nouveau like).
- Unicité du like par utilisateur (contrainte d'unicité en BDD + contrôle applicatif).
- Introduction d'un rôle administrateur pour la modération des articles et des commentaires.
- Suivi d'auteurs (abonnement) et fil d'actualité personnalisé.
- Amélioration du système de recherche (recherche plein texte sur titres et contenus).